(for example) use preview versions of .NET in Azure App Service without needing the preview version to be supported.

Another advantage of SCDs is for regulated industries that require certification or procedure to change applications. In RDDs (such as in Azure App Service) the underlying runtime may be patched at any time without your intervention, potentially leading to noncompliance. With SCDs, your app contains a fixed runtime and can be considered an immutable snapshot of your app. Of course, that means you must make sure to patch the runtime of your SCDs manually, performing regular deployments. Patch versions of the .NET runtime are generally released every month, so make sure to plan for at least monthly releases of your SCD apps.

In this book I discuss RDDs only for simplicity, but if you want to create an SCD, provide a runtime identifier (in this case, Windows 10 64-bit) when you publish your app:

```
dotnet publish -c Release -r win10-x64 --self-contained -o publish_folder
```

The output will contain an .exe file, which is your application, and a ton of .dlls (about 100 MB of .dlls for a default sample app), which are the .NET 7 framework. You need to deploy this whole folder to the target machine to run your app. Note that you need to publish for a specific operating system and architecture. The list of available runtime identifiers is available in the documentation at http://mng.bz/Aolp.

In .NET 7 it's possible to trim these assemblies during the publish process, but this comes with risks in some scenarios. You can also bundle this folder into a single file automatically for easier deployments. For more details, see Microsoft's ".NET application publishing overview" documentation at https://learn.microsoft.com/dotnet/core/deploying.

We've established that publishing your app is important for preparing it to run in production, but how do you go about deploying it? How do you get the files from your computer onto a server so that people can access your app? You have many, many options, so in the next section I'll give you a brief list of approaches to consider.

27.1.2 Choosing a deployment method for your application